Two changes for Chiron

What chiron-mcp, the Ask tab, asks of is4r-chiron and is4r-chiron-ui. For the Chiron developers. 3 November 2026.

In short

- Ask (chiron-mcp) runs beside an unchanged Chiron. Two things it needs are not in Chiron yet: **a pair of fixes in the query engine** (is4r-chiron) and **a build-time switch for the Ask tab** in the React UI (is4r-chiron-ui).
- Both are ready as patch files in the chiron-mcp repository, `askpatch/chiron-fixes.patch` and `askpatch/ui-ask-tab.patch`, and this note carries their diffs. Nothing has been opened against the cchmc repositories; that is for whoever the team decides.
- Ask applies the engine fixes inside its own process at run time, so Ask already works on any Chiron 6.5.x. Chiron's *own* query builder gets the relative-date fix only once Chiron itself carries it: today a "within N days" relationship on a dataset whose event dates are multivalue fails in the Chiron UI.
- For people who do not use Ask these changes do two things: a crash becomes a working query, and a silent overwrite of a cohort entry stops. The UI change is invisible unless the build names an Ask address.

1. Chiron's query engine (is4r-chiron 6.5.4)

Three files, two fixes. The diff is on the next pages; it applies to a checkout with `git apply chiron-fixes.patch` (or `patch -p1 < chiron-fixes.patch`).

1a. A shared default argument in CohortDefProcessor._generate_cd_entry_template

File. `chiron/processors/abstract/cohort_def_processor.py`

What happens. The method is declared with `additional_args={}`, writes `entry_id` and `concept_id` into that argument and returns it. A default argument is created once, so every cohort entry built without arguments in one process is the *same* dict: building a second filter silently rewrites the first, including its values, its exclude flag and its entry id. A web server or any long-running process that builds more than one entry sees it; a one-shot script does not, which is why it went unnoticed.

The fix. `additional_args=None`, and a fresh `dict(additional_args or {})` on every call. The one behaviour change: a caller's own dict is copied, not written into. chiron-mcp had worked around the bug by deep-copying around the call; the fix makes every caller safe.

How it was found. Two filters built in one Ask process came back as one; README, "A Chiron bug this project found".

1b. The relative-date rule and event dates kept in lookup tables

Files. `chiron/query_engine/postgres/sql_alchemy_queries/longitudinal_combos.py` and `.../sections/where_builder.py`

What happens. `LongitudinalCombo.generate_sql_alchemy_clauses_for_event` reads each event date with `getattr(table.c, concept.permanent_id)`, the column a single-value concept has on its collection table. When a collection's event date field is a *multivalue* concept, its values live in a lookup table (`lkp_…`) and the collection table has no such column, so the query builder fails with `AttributeError: <date concept>` as soon as a cohort relates two collections in time ("within 90 days after"). Datasets loaded with every concept multivalue are common: the Synthea bundles are, and so is the synthea-10k on the genoquery instance that John set up as longitudinal. The failure is in the engine, so it is the same in Chiron's own query builder.

The fix. A new method `WhereStatementBuilder.get_event_date_column(oConcept, table, subquery_type, subquery_id)` returns the column that holds an event date concept for a table:

- a single-value concept: the collection table's column, exactly as before;
- a multivalue concept inside a subquery: the lookup alias the alias manager registered for that subquery and concept, joined the way the filters' own lookups are;
- a multivalue concept on a table with no subquery of its own (the outer table of a one-sided combo): a correlated `select … limit 1` of that record's value from the lookup table.

`generate_sql_alchemy_clauses_for_event` takes an optional `column_for(index, table, oConcept)` resolver and the where builder passes this method; without the argument the method behaves as it did, so no other caller changes. The `limit 1` matters only if a multivalue date concept held several values for one record; an event date holds one.

Verified. Eight timed cohorts on synthea-10k (metformin within 90 days after a type 2 diabetes record, an ER visit within 30 days before an asthma record, two records of one table, either side, from the other record's end date, a follow-up narrowing the timed group, an "any of" with a timed condition) count exactly what raw SQL on the warehouse counts (`tests/within_live.py`), on a Mac and on the genoquery instance on 3 November 2026 (179 and 89 for the first two, on the live site). The offline test `tests/chiron_patches.py` covers both fixes without a database. Through Ask, 21 timing questions on the 100,000-patient benchmark set were all answered correctly by Haiku 5.5.

The diff

```
--- a/chiron/processors/abstract/cohort_def_processor.py
+++ b/chiron/processors/abstract/cohort_def_processor.py
@@ -298,7 +298,7 @@
     #         data["unique_patient_count"] = value_list[0]["unique_patient_count"]
     #     return data

-    def _generate_cd_entry_template(self, additional_args={}):
+    def _generate_cd_entry_template(self, additional_args=None):
         """
         Generates a universal base that can be used by self.generate_cohort_def_entry()

@@ -307,7 +307,11 @@
         :return: a cd_entry with concept_id and entry_id set
         :rtype: dict
         """
-        response = additional_args
+        # A fresh dict every call. This used to be a mutable default argument
+        # (additional_args={}), so every cd_entry built without arguments in the same
+        # process was the *same* dict: building a second filter silently rewrote the
+        # first, including its exclude flag and its entry_id.
+        response = dict(additional_args or {})
         response["entry_id"] = helpers.generate_entry_id()
         response["concept_id"] = self.concept.permanent_id
         if self.prefilter_value:
```

```
--- a/chiron/query_engine/postgres/sql_alchemy_queries/longitudinal_combos.py
+++ b/chiron/query_engine/postgres/sql_alchemy_queries/longitudinal_combos.py
@@ -88,11 +88,16 @@
             self.combo[0]["entry_id"] + self.combo[1]["entry_id"] + str(outer_table) + str(index)
         )

-    def generate_sql_alchemy_clauses_for_event(self, table0, table1):
+    def generate_sql_alchemy_clauses_for_event(self, table0, table1, column_for=None):
         """

         :param table0: the sqlalchemy table or alias for the first related collection
         :param table1: the sqlalchemy table or alias for the second related collection
+        :param column_for: optional, column_for(index, table, oConcept) returning the column
+          holding that event date concept for the table of combo index 0 or 1. The default
+          reads it off the collection table, which is where a single-value concept lives; a
+          multivalue date concept lives in a lookup table, and the where builder passes a
+          resolver that joins it (chiron-mcp: event dates on datasets loaded as multivalue).
         :return:
         """
         clauses = []
@@ -101,20 +106,25 @@
         reference_date = self.combo[0]["event_rule"]["reference_date"]
         range_start_target = self.combo[0]["event_rule"]["range_start_target"]
         range_end_target = self.combo[0]["event_rule"]["range_end_target"]
-        date_concept0 = self.combo[0]["collection"].event_date_field.permanent_id
+        collection0 = self.combo[0]["collection"]
+        collection1 = self.combo[1]["collection"]
+        date_concept0 = collection0.event_date_field
         if reference_date == "end":
-            date_concept0 = self.combo[0]["collection"].event_end_date_field.permanent_id
-        date_concept1_range_start = self.combo[1]["collection"].event_date_field.permanent_id
-        date_concept1_range_end = self.combo[1]["collection"].event_date_field.permanent_id
+            date_concept0 = collection0.event_end_date_field
+        date_concept1_range_start = collection1.event_date_field
+        date_concept1_range_end = collection1.event_date_field
         if range_start_target == "end":
-            date_concept1_range_start = self.combo[1][
-                "collection"
-            ].event_end_date_field.permanent_id
+            date_concept1_range_start = collection1.event_end_date_field
         if range_end_target == "end":
-            date_concept1_range_end = self.combo[1]["collection"].event_end_date_field.permanent_id
-        date_column0 = getattr(table0.c, date_concept0)
-        date_column1_range_start = getattr(table1.c, date_concept1_range_start)
-        date_column1_range_end = getattr(table1.c, date_concept1_range_end)
+            date_concept1_range_end = collection1.event_end_date_field
+        if column_for is None:
+
+            def column_for(index, table, oConcept):
+                return getattr(table.c, oConcept.permanent_id)
+
+        date_column0 = column_for(0, table0, date_concept0)
+        date_column1_range_start = column_for(1, table1, date_concept1_range_start)
+        date_column1_range_end = column_for(1, table1, date_concept1_range_end)
         clause = date_column0 - date_column1_range_start >= range_start_days_from_target
         clauses.append(clause)
         clause = date_column0 - date_column1_range_end <= range_end_days_from_target
--- a/chiron/query_engine/postgres/sql_alchemy_queries/sections/where_builder.py
+++ b/chiron/query_engine/postgres/sql_alchemy_queries/sections/where_builder.py
@@ -586,10 +586,37 @@
                 combo.get_criteria_set(1), subquery_type, subquery_id1
             )
             clauses += clauses2
-        date_clauses = combo.generate_sql_alchemy_clauses_for_event(table0, table1)
+        subquery_ids = (subquery_id0, subquery_id1)
+
+        def column_for(index, table, oConcept):
+            return self.get_event_date_column(oConcept, table, subquery_type, subquery_ids[index])
+
+        date_clauses = combo.generate_sql_alchemy_clauses_for_event(table0, table1, column_for)
         clauses += date_clauses
         return clauses
```

```
+    def get_event_date_column(self, oConcept, table, subquery_type, subquery_id):
+        """The column holding an event date concept for `table` (a collection table or alias).
+
+        A single-value concept is a column of the collection table. A multivalue concept
+        lives in its lookup table: inside a subquery that is the lookup alias registered for
+        the subquery, which the caller joins like the aliases of the filters (get_aliases
+        "lookup"); for a table with no subquery of its own (the outer table of a one-sided
+        combo), a correlated lookup of that record's value.
+        """
+        if not oConcept.multivalue:
+            return getattr(table.c, oConcept.permanent_id)
+        if subquery_id is not None:
+            alias = self.alias_manager.get_concept_alias(subquery_type, subquery_id, oConcept)
+            return getattr(alias.c, oConcept.permanent_id)
+        lookup = self.query_info.get_sql_alchemy_table_for_concept(oConcept).alias()
+        return (
+            select(getattr(lookup.c, oConcept.permanent_id))
+            .where(lookup.c._collection_id == table.c._id)
+            .limit(1)
+            .scalar_subquery()
+        )
+
     def get_clauses_related_to_collection(self, oCollection, entry_id, aggregation_criteria_set):
         """This one is for select statement.
```

What chiron-mcp does meanwhile

`chiron_mcp/chiron_patches.py` looks at the host's code right after Django starts and, only where a fix is missing, replaces the method in memory of the Ask process: the first by the default argument's type, the second by whether the where builder has `get_event_date_column`. Nothing on disk is written and Chiron's own web app is untouched. A Chiron that carries the fixes is left alone, so once they are merged the run-time patch stands down by itself.

2. The React UI (is4r-chiron-ui)

One file, `src/overrideConfig.tsx`, about ten lines.

Today. The develop branch already has the Ask tab: a route `ask` under the dataset and an **Ask** entry in the header, with the Ask server's address fixed in the source as `http://localhost:8900`. Every build shows the tab, and it only works on a developer's machine with the server on that port.

The change. The address comes from the build: `VITE_ASK_URL`. Unset, the route and the tab are not added and the UI is as it was. `/ask` means the site's web server serves Ask on the UI's own address, which is what lets Ask read the Chiron login cookie; a full URL is for development. A trailing slash is trimmed.

Verified. On 3 November 2026 the stock UI built without the variable showed no Ask tab (header: Dataset Home, Query, Results, Reports, Aggregate) and, rebuilt with `VITE_ASK_URL=/ask`, showed it, in a browser against a stock Chiron 6.5.4. The chiron-mcp deploy bundle builds its UI with `VITE_ASK_URL=${BASE_PATH}/ask`.

The diff

```
--- a/src/overrideConfig.tsx
+++ b/src/overrideConfig.tsx
@@ -10,8 +10,12 @@
 } from "@mui/icons-material";
 import { useParams } from "react-router-dom";

-// Where the "Ask Chiron" server is running (python -m chiron_mcp.webapp).
-const ASK_URL = "http://localhost:8900";
+// Where the "Ask Chiron" server (chiron-mcp) is served from, set when the UI is built:
+//   VITE_ASK_URL=/ask                    a web server serves it on this same address,
+//                                        which is what lets it read the Chiron login
+//   VITE_ASK_URL=http://localhost:8900   local development, straight at the server
+// Unset, there is no Ask tab and nothing else changes.
+const ASK_URL = (import.meta.env.VITE_ASK_URL || "").replace(/\/$/, "");

 /**
  * Natural-language question box for the current dataset.
@@ -45,7 +49,7 @@
 export const overrideConfig = {
   // apiBaseUrl: "http://localhost:8000",
   routes: {
-    datasetMore: [{ path: "ask", element: <AskRoute /> }],
+    datasetMore: ASK_URL ? [{ path: "ask", element: <AskRoute /> }] : [],
   },
   header: {
     // Chiron's default is "/login/", which Django does not serve; the real form is
@@ -58,7 +62,7 @@
       { label: "Results", link: "/results", icon: <TableChartIcon /> },
       { label: "Reports", link: "/reports", icon: <FileCopyIcon /> },
       { label: "Aggregate", link: "/aggregate", icon: <AddChartIcon /> },
-      { label: "Ask", link: "/ask", icon: <AskIcon /> },
+      ...(ASK_URL ? [{ label: "Ask", link: "/ask", icon: <AskIcon /> }] : []),
     ],
   },
 };
```

Apply with `git apply ui-ask-tab.patch` in an is4r-chiron-ui checkout, then `VITE_ASK_URL=/ask npm run build`, or export it before `npm run dev`.

3. After merging

	Now	Once merged
Chiron's own query builder	A timed relationship on a multivalue-date dataset fails.	Works, on every Chiron that upgrades.
Ask on a deployment's Chiron	Works; the run-time patch supplies the fixes.	Works; the run-time patch finds nothing to do.
The Ask tab in the UI	Apply ui-ask-tab.patch, then build with VITE_ASK_URL.	Build with VITE_ASK_URL; no patch.
A Chiron or UI upgrade	A patch may stop applying and needs a look.	Nothing to do.

Where everything is

What	Path in github.com/ri-ds/chiron-mcp
The engine patch, with this explanation at its top	askpatch/chiron-fixes.patch
The UI patch	askpatch/ui-ask-tab.patch
The run-time version of the engine fixes	chiron_mcp/chiron_patches.py
The fixed sources, as chiron-mcp ships them	vendor/is4r-chiron/… and vendor/is4r-chiron-ui/src/overrideConfig.tsx
Tests	tests/chiron_patches.py (offline), tests/within_live.py (against a warehouse), tests/within_clause.py
The Ask add-on that depends on them	askpatch/README.md